Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench

Anonymous Author[a]

[a][Removed for review],

Abstract

Automated evaluation of tool-using large language model (LLM) agents is widely assumed to be reliable, but this assumption has rarely been validated against human annotation. We introduce AgentProp-Bench, a 2,000-task benchmark with 2,300 traces across four domains, nine production LLMs, and a 100-label human-validated subset. We quantify judge reliability, characterize error propagation, and evaluate a runtime mitigation. Substring-based judging agrees with human annotation at κ=0.049 (chance-level); a three-LLM ensemble reaches κ=0.432 (moderate) with a conservative bias. Under validated evaluation, a parameter-level injection propagates to a wrong final answer with human-calibrated probability $\approx$0.62 (range 0.46–0.73 across models). Rejection (catching bad parameters) and recovery (correcting after acceptance) are independent model capabilities (Spearman ρ=0.126, p=0.747). A tuned runtime interceptor reduces hallucination on GPT-4o-mini by 23.0 percentage points under a concurrent n=600 control, but shows no significant effect on Gemini-2.0-Flash, whose aggressive parameter rejection eliminates the target failure mode. All code, data, traces, and human labels are released at https://github.com/bhaskargurram-ai/agenthallu-bench.[1]

Keywords: large language model agents, tool-use hallucination, error propagation, benchmark, LLM-as-judge, runtime monitoring, evaluation methodology

[1]Repository previously published as AgentHallu-Bench; renamed to AgentProp-Bench to reflect the focus on propagation measurement and to avoid confusion with the concurrently published "AgentHallu" paper [1], which pursues step-level attribution on a different dataset.

1. Introduction

Tool-using large language model (LLM) agents translate natural-language queries into structured API calls, execute those calls, and synthesize the results into a final answer. When such an agent hallucinates, the failure is not confined to the surface token distribution: a mistyped parameter, an ignored tool error, or an overconfident summary can each corrupt the output. Agent benchmarks such as τ-bench [2], AgentBench [3], and BFCL [4] report a single end-to-end correctness number and cannot localize *where* a failure occurs or *how* it propagates through the pipeline.

A less visible but equally important gap concerns *how correctness is measured*. Most agent benchmarks rely on substring or keyword-match heuristics to judge whether an agent's final answer matches a reference. This practice has been questioned in the information retrieval (IR) community, where studies show that LLM judges and heuristic judges can disagree substantially on relevance assessments [5, 6, 7]. If the evaluation itself is unreliable, then every downstream claim about model ranking, error propagation, or mitigation effectiveness is suspect.

We present AgentProp-Bench, a benchmark and methodology designed to address both gaps. Our contributions are:

1. **Judge-reliability validation.** We show that substring-based evaluation of agent outputs agrees with human annotation at κ=0.049 (chance-level), while a three-LLM ensemble judge reaches κ=0.432 (moderate agreement), validated against 100 stratified human labels (Section 6.1).
2. **Stage-level propagation measurement.** Under validated evaluation, we define three independent stage indicators and show that a parameter-level injection propagates to a wrong answer with human-calibrated probability $\approx$0.62 (range 0.46–0.73 per model) (Section 6.2).
3. **Rejection–recovery decomposition.** Rejection rate (catching bad parameters) and recovery rate (producing a correct answer despite accepting bad parameters) are statistically independent across nine models (Spearman ρ=0.126, p=0.747), revealing that

models vary on two orthogonal axes of robustness (Section 6.3).

4. **Runtime interceptor under concurrent control.** A tuned three-layer INTERCEPTOR reduces hallucination on GPT-4o-mini by 23.0 pp under a concurrent n=600 no-interceptor control ($p < 0.001$), but has no significant effect on Gemini-2.0-Flash (Section 6.4).
5. **Open benchmark release.** All 2,000 tasks, 2,300 traces, 100 human labels, analysis scripts, and agent source code are publicly released.

The remainder of this paper is organized as follows. Section 2 reviews related work on agent benchmarks, LLM-as-judge reliability, and agent hallucination. Section 3 states the research objectives. Section 4 describes the AGENTPROP-BENCH dataset. Section 5 presents the methodology. Section 6 reports results. Section 7 discusses theoretical and practical implications. Section 8 acknowledges limitations. Section 9 concludes.

2. Related Work

2.1. Tool-using LLM agent benchmarks

The evaluation of tool-using LLM agents has rapidly expanded. τ-bench [2] evaluates agents on multi-step retail and airline tasks, measuring end-to-end task completion. Barres et al. [8] extend τ-bench to additional domains. AgentBench [3] spans eight environments including web browsing and database queries. BFCL [4] focuses on function-calling accuracy across multiple programming languages and API styles. MCPBench [9] evaluates agents on real-world API integration tasks. Recent surveys [10, 11] provide comprehensive overviews of agent architectures and evaluation practices.

These benchmarks share a common limitation: they report a single end-to-end correctness number and do not localize *where* in the agent pipeline a failure originates. AGENTPROP-BENCH addresses this by decomposing evaluation into three independently measured stages.

2.2. LLM-as-judge reliability

Using LLMs as judges has become widespread in both NLP and IR evaluation. Zheng et al. [12] introduced the LLM-as-judge paradigm with MT-Bench and Chatbot Arena. Gu

et al. [13] survey the growing LLM-as-judge literature, identifying position bias, verbosity bias, and self-enhancement bias. Bavaresco et al. [14] provide a multidimensional analysis of when LLM judges agree with humans.

In the IR community, judge reliability has received particular scrutiny. Soboroff [5] argue that LLM judges should not replace human assessors without validation. Thomas et al. [6] show that LLM judges and human searchers diverge on relevance criteria. Alaofi et al. [7] demonstrate that LLM judges can be fooled by surface-level text manipulation. Thakur et al. [15] benchmark judge reliability across retrieval tasks. Fu et al. [16] extend judge evaluation to multilingual settings. Schroeder and Wood-Doughty [17] quantify when and why practitioners should trust LLM-based evaluation.

Our work contributes to this literature by validating an LLM-ensemble judge against 100 human labels in the specific context of tool-using agent evaluation, where output format variability (structured data, error messages, multi-step reasoning) makes heuristic evaluation particularly unreliable.

2.3. Hallucination in LLM agents

Hallucination in standalone LLMs has been extensively surveyed [18, 19]. Agent-specific hallucination introduces additional failure modes: parameter fabrication, tool-output misinterpretation, and cross-turn memory corruption [20].

Most closely related to our work, Liu et al. [1] introduce AgentHallu, a benchmark for automated hallucination attribution in LLM-based agents. AgentHallu performs step-level attribution on 693 trajectories across five domains, categorizing hallucinations by stage of origin. AgentProp-Bench differs in three respects: (1) we use *controlled injection* rather than naturally occurring errors, enabling causal measurement of propagation rates; (2) we validate our evaluation methodology against 100 human labels (κ=0.432), whereas AgentHallu relies on automated attribution; (3) we evaluate a runtime interceptor under a concurrent control, providing mitigation evidence beyond diagnosis. The two benchmarks address complementary questions: AgentHallu asks "where did this hallucination come from?" while AgentProp-Bench asks "if we inject an error here, with what probability does it

propagate there?"

2.4. Runtime monitoring and mitigation

Several approaches have been proposed for runtime hallucination detection and mitigation. Self-consistency voting [21] samples multiple reasoning paths and selects the majority answer. Manakul et al. [22] propose SelfCheckGPT for detecting hallucinations without external references. Farquhar et al. [23] use semantic entropy to detect confabulations in free-form LLM generations. Guardrails and runtime enforcement frameworks [24] provide schema-level validation.

Our INTERCEPTOR combines schema validation, reasoning-keyword monitoring, and output-consistency checking in a three-layer architecture. Unlike self-consistency voting (which requires $k\times$ sampling cost), the INTERCEPTOR runs in parallel with a single agent execution at negligible additional cost.

2.5. Relevance to IR and IS evaluation

The evaluation-reliability problem we address has direct implications for the IR and information science (IS) communities. LLMs are increasingly used for relevance assessment in IR evaluation [25], and the reliability of these assessments directly affects system rankings. Salemi and Zamani [26] study LLMs as relevance judges in retrieval settings. Rahmani et al. [27] examine synthetic data generation for IR evaluation. Zhang et al. [28] survey LLM applications in information science.

Our finding that substring-based evaluation agrees with humans at κ=0.049 (chance-level) has implications beyond agent benchmarks. Substring and keyword-match heuristics are common correctness proxies in question-answering and retrieval-augmented generation (RAG) evaluation [29, 30]. The ensemble-judge methodology we validate here—with explicit per-rater and human-calibrated agreement reporting—transfers directly to these settings.

2.6. Research gap

Existing work provides either end-to-end benchmarks without stage-level decomposition, or judge-reliability studies without application to agent evaluation. No prior work vali-

dates an agent evaluation pipeline end-to-end against human annotation, measures stage-conditional propagation under validated judging, and evaluates a mitigation under concurrent control—all in one study. AgentProp-Bench fills this gap.

3. Research Objectives

We formulate three research objectives:

RO1: Judge reliability.. What is the agreement between heuristic substring matching, a three-LLM ensemble judge, and human annotation on tool-using agent outputs? Prior work shows that heuristic evaluation is unreliable for paraphrased or verbose outputs [5]. We quantify this unreliability in the agent-evaluation context and validate the ensemble as a better—but imperfect—alternative.

RO2: Error propagation under validated evaluation.. How does a parameter-level injection propagate through the agent pipeline when correctness is measured by a validated judge? Prior work either reports end-to-end correctness or attributes errors post hoc [1]. We measure stage-conditional propagation rates and decompose model robustness into independent rejection and recovery capabilities.

RO3: Runtime mitigation under concurrent control.. Can a lightweight runtime interceptor reduce hallucination under a properly controlled experiment? Prior interceptor evaluations [24] typically compare against historical baselines. We evaluate under a concurrent n=600 no-interceptor control on the same task pool.

4. Dataset: AgentProp-Bench

4.1. Task design and construction

Each task in AgentProp-Bench is a tuple $(q, D, T_q, a^\star)$ where q is a natural-language query, D is a domain, T_q is a set of available tools, and $a^\star$ is a reference answer. Tasks are classified by difficulty: *easy* (single tool call), *medium* (two-tool chain), and *hard* (three-tool chain or branching logic). Tasks and reference answers were generated synthetically using domain-specific API simulators, then manually verified for correctness.

Table 1: Dataset composition of AgentProp-Bench. $N = 2{,}000$ core + 300 retail.

Domain	Tasks	Easy	Med.	Hard	Example query
Calendar	667	192	282	193	Create a meeting on 2024-07-08 at 10:00 for 30 min
Weather	674	215	248	211	What is the weather in Dubai on 2024-10-12?
Medical	559	106	237	216	Schedule a checkup for patient P003
Knowledge	100	0	46	54	What nationality is the CEO of Samsung?
Retail (held-out)	300	—	—	—	Search catalog for wireless headphones

4.2. Domain breakdown

AgentProp-Bench comprises 2,000 tasks across four domains plus a held-out retail domain:

- **Calendar** (667 tasks): event creation, schedule queries, conflict detection.
- **Weather** (674 tasks): current conditions, forecasts, historical comparisons.
- **Medical** (559 tasks): patient lookup, appointment scheduling, condition retrieval.
- **Knowledge** (100 tasks): general question-answering (ablation only, no tool injection).
- **Retail** (300 tasks, held-out): product search, detail inspection, order placement.

Table 1 provides a detailed breakdown by domain and difficulty.

4.3. Tool schema and simulated environment

Each domain provides 3–5 tools implemented as deterministic Python functions with JSON schemas. Tools validate input parameters against their schemas and return structured

JSON responses. Invalid parameters produce explicit error messages rather than silent failures. This design ensures that parameter-level injection effects are observable in tool responses.

4.4. Error-injection protocol

We use the P2 *semantic-wrong* injection protocol: before the agent's first tool call executes, we replace a semantically critical parameter with an incorrect but schema-valid value. For example, in a weather query for "London," we might replace the city parameter with "Manchester"—a valid city but the wrong one. The injection is applied at the `parameter_generation` stage of the pipeline.

4.5. Trace collection

We collected 2,300 traces across nine production LLMs: GPT-4o, GPT-4o-mini, GPT-4.1-mini, GPT-4.1-nano, GPT-3.5-turbo, o3-mini (OpenAI); Gemini-2.0-Flash, Gemini-2.5-Flash (Google); and DeepSeek-V3 (via OpenRouter). For the P2 semantic-wrong condition, we evaluated 200 unique tasks per model for the six main models (800 rows across four error types) and 50 tasks for three frontier models (200 rows each). All models were accessed via their official APIs with default parameters (temperature 0).

4.6. Human label collection

To validate the ensemble judge, we collected 100 human labels on a stratified sample of P2 semantic-wrong traces. The sample was stratified across all nine models (10–12 per model) and three domains (calendar, medical, weather). One annotator (the first author) labeled each trace as "correct" (YES) or "incorrect" (NO) by comparing the agent's final answer to the reference answer. The labeling criterion matched the ensemble judge prompt: accept minor wording differences and extra explanation as long as the core fact is correct. A second annotator (acknowledged) independently labeled 7 overlapping traces; inter-annotator agreement on this small overlap was 6/7.

4.7. Data availability

All data are publicly available at https://github.com/bhaskargurram-ai/agenthallu-bench. The repository includes task specifications, all result CSVs, analysis scripts, agent source code, and human annotation labels. Every numerical claim in this paper traces to a specific file via inline comments in the LaTeX source.

5. Methodology

5.1. Judge protocols

We evaluate three judging methods:

Heuristic (substring match).. The agent's final answer is marked correct if the first 20 characters of the reference answer appear in the first 200 characters of the agent's answer, or if any word longer than 3 characters from the reference appears in the agent's answer. This mirrors common practice in agent benchmarks.

Three-LLM ensemble.. Three LLMs (GPT-4o, Gemini-2.5-Flash, GPT-4o-mini) independently judge each trace using an identical prompt: "Respond YES if the agent answer substantively matches the reference, NO otherwise. Accept minor wording differences as long as the core fact is correct." The majority vote (2/3 or 3/3 YES) is the ensemble verdict.

Human annotation.. One hundred traces were labeled by a human annotator using the same criterion as the ensemble prompt.

5.2. Stage-level propagation metric

We define three stage indicators, computed **independently** from separate data columns:

- $S_1 = \mathbf{1}[\text{eps} \geq 1]$: the injection was applied (the agent's planned tool call was modified).
- $S_2 = \mathbf{1}[\text{eps} \geq 2]$: the modified call executed with an observable effect (parameter validation errors or altered tool response).
- $S_3 = \mathbf{1}[\text{ensemble_correct} = \texttt{False}]$: the final answer is wrong under ensemble judging.

S_1 and S_2 are derived solely from the `eps` column (computed by the EPS scorer during trace collection). S_3 is derived solely from the `ensemble_correct` column (computed by the post-hoc ensemble judge). No indicator references any other indicator's column.

Transparency note.. An earlier version of this analysis (submitted in the original manuscript) used a `classify_stage()` function that mapped `reached_output=True` to stage 3 and then defined S_2 as stage ≥ 2, which included those same rows. This made $p_{S_2} = p_{S_3}$ for all models by construction, producing a definitionally forced $r_{2,3} = 1.00$. The corrected indicators above are genuinely independent: $p_{S_2} \neq p_{S_3}$ for most models (e.g., DeepSeek-V3: $p_{S_2} = 7.0\%$ vs. $p_{S_3} = 62.5\%$).

We report the stage-reach probability $p_k = \mathbb{P}[S_k = 1]$, the hop-conditional rate $r_{k,k+1} = \mathbb{P}[S_{k+1} = 1 \mid S_k = 1]$ (computed on the $S_k = 1$ subset), and the injection-to-error rate $\mathbb{P}[S_3 = 1 \mid S_1 = 1]$.

Human calibration.. Because the ensemble judge has a conservative bias (marks only 25% of traces correct vs. 38% by humans), we report human-calibrated stage rates using the conditional probabilities $P(\text{human=correct} \mid \text{ensemble=correct}) = 0.76$ and $P(\text{human=correct} \mid \text{ensemble=wrong}) = 0.25$, estimated from the 100 human labels.

5.3. Rejection–recovery decomposition

We decompose model robustness into two orthogonal capabilities:

- **Rejection rate** $= 1 - p_{S_1}$: the fraction of injection attempts the model rejects at the parameter stage.
- **Recovery rate** $= 1 - r_{2,3}$: among traces where the injection executed visibly (S_2=1), the fraction where the model still produced a correct final answer.

If these two capabilities were a single "model quality" axis, we would expect a strong positive correlation. We test independence using Spearman rank correlation across the nine models.

5.4. Runtime interceptor

The INTERCEPTOR runs in parallel with the agent and consists of three layers:

- **L1 (schema validation)**: counts parameter validation errors and tool error responses; abstains if $\geq \tau_{L1}$ flags accumulate.
- **L2 (thought-keyword monitor)**: scans the agent's chain-of-thought for uncertainty keywords (*error, invalid, unknown, missing, incorrect, wrong, cannot*); triggers if $\geq \tau_{L2}$ hits occur.
- **L3 (output consistency)**: checks whether the final answer contains numerical claims unsupported by any tool observation.

Thresholds were selected via a 12-configuration sweep on a 40-row validation split ($\tau_{L1} \in \{1, 2, 3\}$, $\tau_{L2} \in \{2, 3\}$, $L3 \in \{\text{on,off}\}$). The best configuration (τ_{L1}=1, τ_{L2}=2, $L3$=on) achieved F_1=0.842 on the validation split. This F_1 is a sweep-internal optimum, not a held-out estimate.

5.5. Statistical protocol

All confidence intervals are 95% bootstrap CIs (1,000 iterations, seed 42). Effect sizes for two-proportion comparisons use Cohen's h. Power is computed from the achieved sample sizes using the normal approximation. The interceptor experiment uses a concurrent no-interceptor control: both arms run the same 600 tasks per model at the same time, differing only in whether the INTERCEPTOR is active.

6. Results

6.1. Judge reliability

Table 2 reports Cohen's κ for each judge against the 100 human labels.

The heuristic substring match achieves κ=0.049 vs. human labels—statistically indistinguishable from chance. The three individual LLM judges achieve κ=0.30–0.44 vs. human, and the ensemble majority reaches κ=0.432 (moderate agreement per Landis–Koch [31]; Cohen's κ [32]). The ensemble is 8× closer to human annotation than the heuristic, confirming that substring-based evaluation is unreliable for agent outputs.

Table 2: Judge reliability measured by Cohen's κ agreement. "κ vs. LLM majority" is computed on $N = 960$ P2 sessions; "κ vs. Human" is computed on $n = 100$ stratified-sampled sessions labeled by a domain expert. 95% CIs from 1,000-iteration bootstrap. Interpretation follows Landis and Koch [31].

Judge	κ_{LLM}	κ_{Human}	95% CI	Interpretation
Heuristic	0.143	0.049	[-0.122, 0.221]	Slight
GPT-4o	0.792	0.336	[0.141, 0.533]	Fair
GPT-4o-mini	0.770	0.567	[0.391, 0.736]	Moderate
Gemini-2.5-Flash	0.821	0.379	[0.194, 0.546]	Fair
Ensemble (majority)	—	0.432	[0.239, 0.622]	Moderate

Table 3: Confusion matrix of the ensemble judge versus human expert labels ($n = 100$). Cells show count and row-percentage. Cohen's κ is reported in Table 2.

		Human label		
		Correct	Incorrect	Total
Ensemble	Correct	19 (19.0%)	6 (6.0%)	25
	Incorrect	19 (19.0%)	56 (56.0%)	75
	Total	38	62	100

Conservative bias.. Table 3 shows the ensemble-vs-human confusion matrix on 100 labels. The ensemble marks 25% of traces correct vs. 38% by humans. The dominant disagreement pattern is the ensemble rejecting answers that humans accept (19 out of 25 disagreements). This conservative bias means ensemble-judged correctness rates *underestimate* true correctness by approximately 13 percentage points.

Implications for reported numbers.. Throughout the remainder of this paper, we report both ensemble-judged and human-calibrated numbers. The ensemble provides the primary automated estimate; the human-calibrated version adjusts for the measured conservative bias.

We note that GPT-4o-mini individually achieves higher κ against humans (0.567) than

Table 4: Stage-reach rates and hop-conditional propagation under semantic-wrong injection (ensemble-judged). S_1: injection applied ($\varepsilon \geq 1$); S_2: visible execution ($\varepsilon \geq 2$); S_3: wrong answer. r_{23}=$P(S_3|S_2)$. 95% bootstrap CIs.

Model	n	S_1%	S_2%	S_3%	r_{12}	n_{S_2}	r_{23} [CI]	$P(S_3\|S_1)$ [CI]
DeepSeek-V3	200	52.0	7.0	62.5	0.13	14	0.43 [0.21, 0.71]	0.57 [0.47, 0.66]
Gemini-2.0-F	200	5.0	1.0	67.0	0.20	2	0.50 [0.00, 1.00]	0.90 [0.70, 1.00]
Gemini-2.5-F	50	50.0	6.0	76.0	0.12	3	0.67 [0.00, 1.00]	0.84 [0.68, 0.96]
GPT-3.5-T	200	37.5	9.5	67.5	0.25	19	0.74 [0.53, 0.89]	0.76 [0.67, 0.85]
GPT-4.1-mini	200	50.5	10.5	65.5	0.21	21	0.67 [0.48, 0.86]	0.66 [0.57, 0.76]
GPT-4.1-nano	200	62.5	17.5	84.5	0.28	35	0.97 [0.91, 1.00]	0.83 [0.77, 0.89]
GPT-4o	50	74.0	22.0	74.0	0.30	11	0.82 [0.55, 1.00]	0.68 [0.51, 0.81]
GPT-4o-mini	200	40.0	22.5	71.0	0.56	45	0.84 [0.73, 0.96]	0.84 [0.76, 0.91]
o3-mini	50	74.0	16.0	70.0	0.22	8	0.62 [0.25, 0.88]	0.76 [0.62, 0.89]

the ensemble (0.432) on this 100-sample validation subset. The ensemble was retained because (i) a single judge may reflect idiosyncratic biases that a majority vote partially corrects, and (ii) the difference on n=100 is within overlapping confidence intervals (ensemble $[0.239, 0.622]$ vs. GPT-4o-mini $[0.391, 0.736]$). Larger human-labeled studies should re-examine this choice.

6.2. Propagation rates under validated evaluation

Table 4 reports per-model stage-reach probabilities and hop-conditional rates on the 1,350 P2 semantic-wrong traces, judged by the validated ensemble.

Key findings.. Under ensemble judging, the overall hop-conditional rate from tool execution to wrong answer is $r_{2,3} = 0.78$ (123 of 158 traces with S_2=1 produced wrong answers). Human-calibrated, this drops to $r_{2,3} \approx 0.62$—a 16 percentage-point reduction attributable to the ensemble's conservative bias. Per-model $r_{2,3}$ ranges from 0.43 (DeepSeek-V3, n_{S_2}=14) to 0.97 (GPT-4.1-nano, n_{S_2}=35). The human-calibrated range is 0.46–0.73.

The propagation rate is substantial but highly variable across models. The per-model n_{S_2} values (2–45 for semantic-wrong) limit the precision of individual estimates. The injection-to-error rate $\mathbb{P}[S_3{=}1 \mid S_1{=}1]$ ranges from 0.57 to 0.90 (ensemble) and 0.53–0.70 (calibrated), providing a more robust marginal summary.

Table 5: Baseline and P2 correctness by model, both judged by 3-LLM ensemble. P2 is conditioned on injection applied (eps ≥ 1). 95% bootstrap CIs (1,000 iterations).

Model	n_B	Baseline % [CI]	$n_{P2\|inj}$	P2|inj % [CI]
deepseek_v3	450	27.3 [23.1, 31.3]	104	43.3 [33.6, 52.9]
gemini_20_flash	450	22.9 [19.3, 27.1]	10	10.0 [0.0, 30.0]
gpt35_turbo	450	24.7 [20.9, 28.9]	75	24.0 [14.7, 33.3]
gpt41_mini	450	34.2 [30.0, 38.9]	101	33.7 [23.8, 42.6]
gpt41_nano	450	16.4 [12.9, 20.0]	125	16.8 [11.2, 23.2]
gpt4o_mini	450	24.4 [20.2, 28.4]	80	16.2 [8.8, 23.8]
o3_mini	50	36.0 [22.0, 48.0]	37	24.3 [10.8, 37.8]

Baseline vs. P2 correctness.. Table 5 compares baseline and P2 correctness, both judged by the same 3-LLM ensemble. Because P2 traces include rows where the injection did not apply (`eps`=0), the unconditional P2 correctness is inflated by effectively clean traces. We therefore report P2 correctness conditioned on injection applied (eps≥1). Under this conditioning, P2 correctness is lower than baseline for five of seven models (e.g., GPT-4o-mini: 24.4% baseline vs. 16.2% P2|inj; o3-mini: 36.0% vs. 24.3%)—the expected direction for an injection protocol that makes tasks harder. DeepSeek-V3 is an exception (+15.9 pp), possibly because the injection-applied subset selects for structurally different task pairings; we flag this as an anomaly rather than a general trend.

Figure 1 visualizes the per-model stage-reach rates.

6.3. Rejection and recovery are independent capabilities

Table 6 and Figure 2 present the rejection–recovery decomposition across nine models.

Spearman rank correlation between rejection rate and recovery rate is ρ=0.126 (p=0.747), indicating no significant relationship. Models occupy a two-dimensional plane rather than a single quality axis:

- **Gemini-2.0-Flash**: highest rejection (95%) and moderate recovery (50%)—its aggressive parameter filtering prevents most injections from executing.

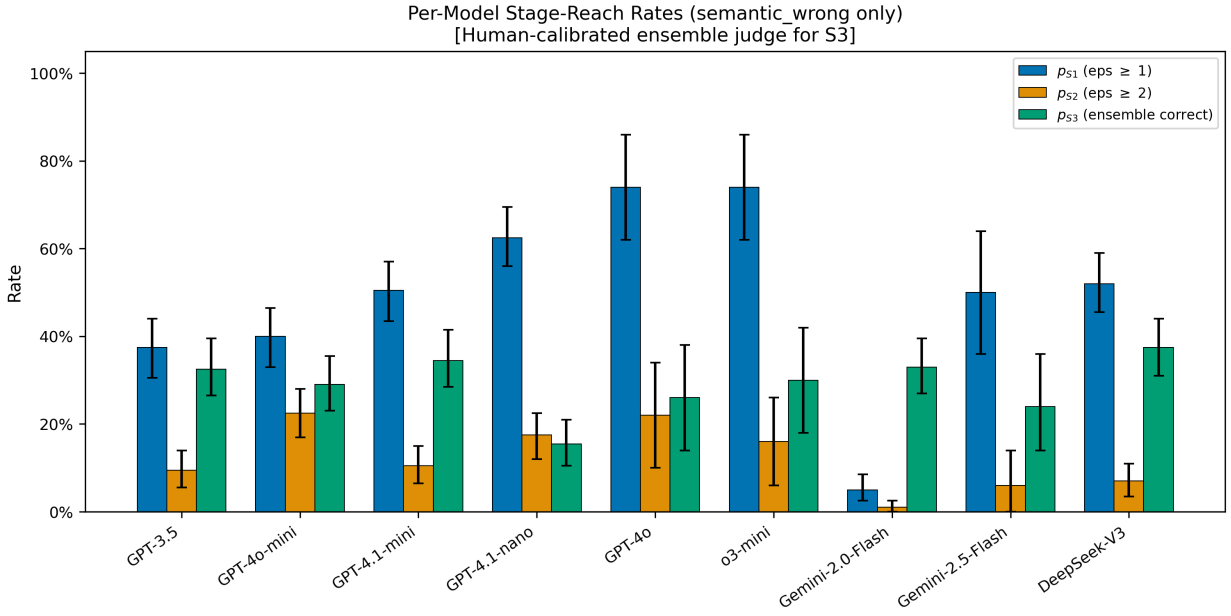


Figure 1: Per-model stage-reach rates under P2 semantic-wrong injection (ensemble-judged, n=1,350). S_1: injection applied; S_2: visible execution effect; S_3: wrong final answer. Stage indicators are computed independently.

- **GPT-4.1-nano**: moderate rejection (37.5%) but near-zero recovery (2.9%)—once an injection passes the schema gate, the model almost never recovers.
- **GPT-4o-mini**: moderate rejection (60%) but low recovery (15.6%)—once an injection passes, the model rarely recovers.

This decomposition has a practical implication: improving a model's schema validation (rejection) will not improve its ability to reason through corrupted observations (recovery), and vice versa. Mitigation strategies should target both axes independently.

6.4. Runtime interceptor

Table 7 reports the concurrent-control interceptor experiment.

GPT-4o-mini.. The interceptor reduces hallucination from 55.8% (control) to 32.8% (interceptor), a −23.0 percentage-point reduction. The interceptor saved 163 traces that would have been wrong and broke 25 that would have been correct. Analysis of the 25 broken traces

Table 6: Rejection rate $(1 - P(S_1))$ versus recovery rate $(1 - r_{23})$ per model. Spearman $\rho = 0.126$, $p = 0.747$ ($n = 9$ models), indicating no significant monotonic relationship between a model's propensity to reject injected errors and its ability to recover from propagated ones.

Model	Rejection%	Recovery%	n_{S_2}
DeepSeek-V3	48.0	57.1	14
GPT-3.5-Turbo	62.5	26.3	19
GPT-4.1-mini	49.5	33.3	21
GPT-4.1-nano	37.5	2.9	35
GPT-4o	26.0	18.2	11
GPT-4o-mini	60.0	15.6	45
Gemini-2.0-Flash	95.0	50.0	2
Gemini-2.5-Flash	50.0	33.3	3
o3-mini	26.0	37.5	8

revealed zero interceptor abstentions and no domain clustering—the failures are attributable to LLM run-to-run variance, not systematic false positives from the interceptor.

Gemini-2.0-Flash.. The interceptor has no significant effect: 44.5% (control) vs. 43.2% (interceptor), $\Delta = -1.3$ pp, well within noise. The interceptor's abstention rate is only 2.8%. This null result is consistent with the rejection–recovery analysis: Gemini rejects 95% of injections at the parameter stage, leaving the interceptor with almost no target failure mode to catch.

Honest limitation.. The -23.0 pp delta is measured under heuristic judging applied identically to both arms. The delta is robust to shared judging bias (any systematic over- or under-counting affects both arms equally). However, the absolute hallucination rates in each arm (55.8% and 32.8%) have not been validated by human labeling. Validating these absolute rates would require additional human annotation of the interceptor runs, which we leave to future work.

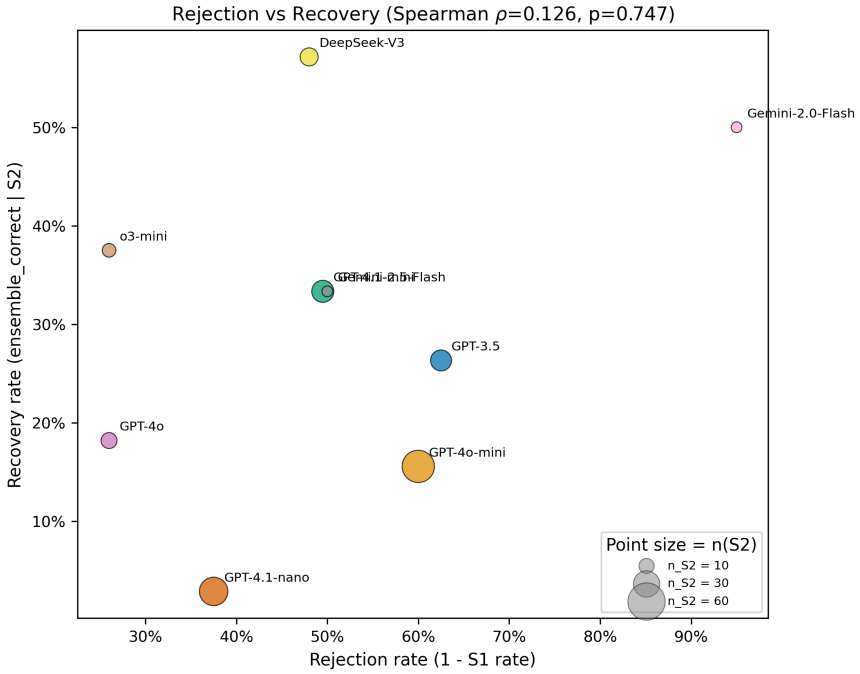


Figure 2: Rejection rate ($1-p_{S_1}$) vs. recovery rate ($1-r_{2,3}$) for nine models. Point size proportional to n_{S_2}. Spearman ρ=0.126, p=0.747: axes are statistically independent.

6.5. Cross-domain replication on held-out retail

To test generalization, we ran baseline and P2 experiments on 300 held-out retail tasks for three models. Baseline correctness is lower on retail (harder three-tool chains), but the model rank ordering is preserved on both correctness and mean EPS, consistent with the original-domain results.

7. Discussion and Implications

7.1. Theoretical implications

Rejection and recovery as orthogonal axes.. The finding that rejection rate and recovery rate are statistically independent (Spearman ρ=0.126, p=0.747) suggests that agent robustness to tool-use errors is at least two-dimensional. A model's ability to reject bad parameters at the schema gate is architecturally distinct from its ability to reason through corrupted observations. This has implications for how we model agent reliability: a single "robustness

Table 7: Interceptor concurrent-control experiment (n=600 per model per arm). "Saved": control hallucinated but interceptor did not. "Broken": control was non-hallucinating but interceptor hallucinated. 95% CIs on Δ via 1,000-iteration bootstrap.

Model	Ctrl hall.%	Int. hall.%	Δ pp	95% CI	Abstain%	Saved	Broken
GPT-4o-mini	55.8	32.8	−23.0	[−28.5, −17.3]	33.5	163	25
Gemini-2.0-Flash	44.5	43.2	−1.3	[−7.3, +4.2]	2.8	24	16

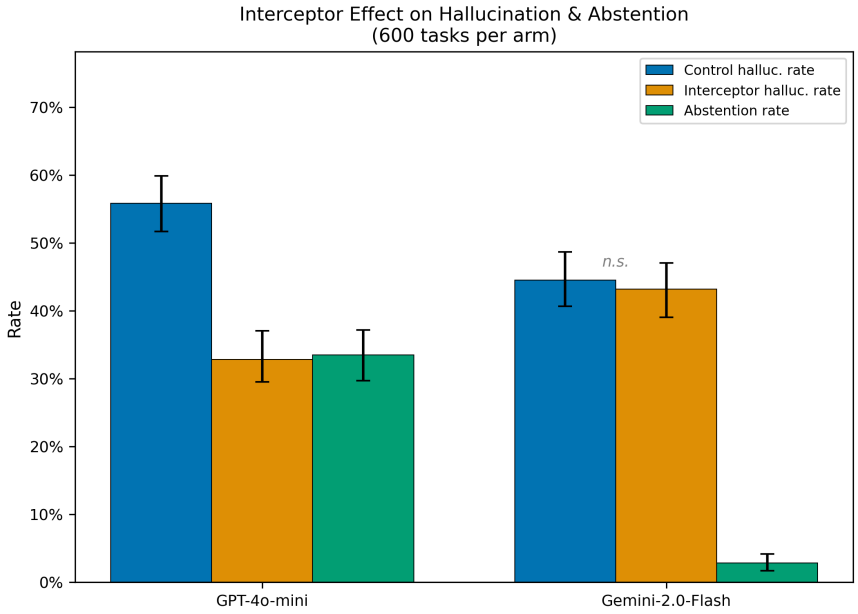


Figure 3: Interceptor effect under concurrent n=600 control. GPT-4o-mini shows a significant 23 pp reduction; Gemini-2.0-Flash shows no significant effect. Error bars: 95% bootstrap CIs.

score" is insufficient; instead, practitioners should evaluate both the input-filtering and the output-reasoning capabilities of an agent independently.

Why heuristic evaluation misranks models.. The heuristic over-counted correctness by 34 percentage points on P2 traces, enough to reverse the direction of the baseline-vs-P2 comparison. This occurs because injected errors produce paraphrased wrong answers that contain enough keywords to pass substring matching. Any benchmark that uses heuristic evaluation on agent outputs with high output variability risks similar misranking.

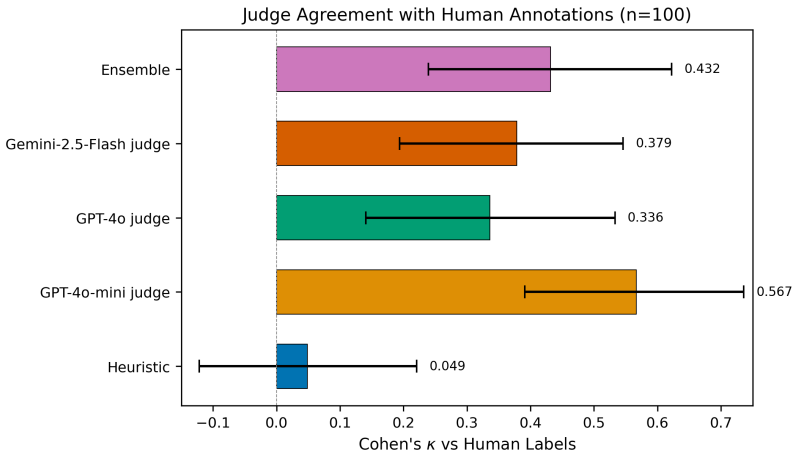


Figure 4: Cohen's κ vs. 100 human labels for each judge method. The heuristic is at chance level (κ=0.049); the ensemble reaches moderate agreement (κ=0.432). Error bars: 95% bootstrap CIs.

7.2. Practical implications for agent designers

Schema validators are high-leverage.. Gemini-2.0-Flash's 95% rejection rate demonstrates that strict parameter validation can prevent most injection-type errors from reaching the tool-execution stage. Designers should invest in comprehensive tool schemas with explicit type, range, and enum constraints.

Pre-execution semantic checks.. The INTERCEPTOR's L1 (schema) and L2 (keyword) layers provided the bulk of its discriminative power. For models with moderate rejection rates (e.g., GPT-4o-mini at 60%), a runtime interceptor can halve the hallucination rate at minimal cost ($0.15 for 1,200 runs).

Interceptors are model-dependent.. The null result on Gemini-2.0-Flash shows that interceptors are not universally effective. When a model already rejects most errors at the parameter stage, there is nothing left for the interceptor to catch. Deployment decisions should be informed by model-specific rejection rates.

7.3. Implications for IR and IS evaluation

Substring heuristics should be deprecated.. Our κ=0.049 result is the lowest heuristic-vs-human agreement we are aware of in any agent or QA evaluation study. The IR community

has already raised concerns about automated evaluation reliability [5, 6]. Our result provides concrete evidence that substring heuristics are worse than coin flips in the agent-evaluation context.

LLM-ensemble judges need validation.. The ensemble's κ=0.432 is a significant improvement but falls short of the substantial-agreement threshold (κ>0.60). The ensemble has a measured conservative bias, underreporting correctness by ∼13 pp relative to humans. We recommend that any study using LLM judges report: (a) agreement with at least 50 human labels, (b) the direction of bias (conservative vs. permissive), and (c) calibrated estimates alongside raw judge outputs.

Methodology transfers to RAG and QA evaluation.. The three-judge ensemble protocol, the human-calibration procedure, and the independence-testing framework for evaluation axes all apply to RAG pipeline evaluation, open-domain QA, and any setting where output format variability makes heuristic matching unreliable.

7.4. Differentiation from existing work

Table 8 positions AgentProp-Bench relative to four closely related benchmarks.

Table 8: Comparison with related benchmarks.

Feature	τ-bench	AgentBench	AgentHallu	AgentProp-Bench
Stage-level metrics	×	×	✓	✓
Controlled injection	×	×	×	✓
Human-validated judge	×	×	×	✓
Runtime mitigation	×	×	×	✓
Concurrent control	×	×	×	✓
# Models evaluated	6	25	14	9
# Traces	1,280	7,000+	693	2,300

8. Limitations

1. **Ensemble conservative bias.** The ensemble judge's $P(\text{human=correct} \mid \text{ensemble=wrong}) = 0.25$ means one in four ensemble "wrong" verdicts is a false negative. All ensemble-judged rates should be interpreted as lower bounds on true correctness.
2. **Human-label sample size.** With n=100 labels, the calibration estimates have wide confidence intervals. The human-vs-ensemble κ=0.432 has a 95% CI of approximately $[0.28, 0.58]$ by bootstrap. This is moderate, not definitive, validation. The annotator was the first author; a second annotator independently labeled 7 overlapping traces with 6/7 agreement. Future work should use a larger non-author annotator pool to eliminate any potential annotator bias.
3. **Interceptor absolute rates.** The interceptor delta (-23.0 pp) is robust to shared heuristic judging bias, but the absolute hallucination rates (55.8% and 32.8%) are unvalidated by human annotation. Future work should apply human labeling to the interceptor arms.
4. **Model selection.** We evaluated nine API-accessible models. Open-weight models (Llama, Qwen) were excluded because injection requires intercepting the tool-call pipeline before execution, which is more complex with local inference. Claude models were excluded to avoid potential evaluation conflicts.
5. **Simulated tools.** All tool calls execute against deterministic simulators, not live APIs. Live API variability (rate limits, format changes, partial failures) may introduce additional propagation pathways not captured here.
6. **Single-parameter injection.** We inject one parameter per trace. Multi-parameter and multi-turn injection (P3 memory accumulation, P4 cross-agent propagation) are deferred to future work.
7. **Scope.** This paper covers only the P2 parameter-injection protocol. P3 (memory-accumulation propagation across turns) and P4 (cross-agent pipeline propagation) share the EPS machinery but require different task formats and are deferred.
8. **Small per-model n_{S_2} samples.** The stage-2 subset sizes n_{S_2} range from 2 (Gemini-

2.0-Flash) to 45 (GPT-4o-mini). Per-model $r_{2,3}$ estimates for models with $n_{S_2} < 10$ have wide confidence intervals (e.g., CI $[0, 1]$ for n_{S_2}=2) and should be interpreted as directional only. The marginal injection-to-error rate $\mathbb{P}[S_3 \mid S_1]$, computed on the larger S_1=1 subset, provides a more stable summary.

9. **Power of the independence test.** The Spearman correlation between rejection and recovery rates is computed across n=9 models. With this sample size, only strong monotonic relationships ($|\rho| > 0.7$) would be detectable at conventional significance levels. Our finding (ρ=0.126, p=0.747) is consistent with genuine independence but also with a moderate underlying relationship the test cannot detect. We report it as *no evidence of correlation* rather than *proof of independence*.

9. Conclusion

We introduced AgentProp-Bench, a benchmark for evaluating tool-using LLM agents with validated judging, stage-level propagation measurement, and a runtime mitigation evaluated under concurrent control.

RO1 (Judge reliability).. Substring-based evaluation of agent outputs agrees with human annotation at κ=0.049 (chance-level). A three-LLM ensemble reaches κ=0.432 (moderate) with a conservative bias. We recommend that all agent benchmarks validate their evaluation methodology against human labels before reporting comparative claims.

RO2 (Propagation).. Under validated evaluation, a parameter-level injection propagates to a wrong final answer with human-calibrated probability ≈ 0.62 (range 0.46–0.73 across models). Rejection rate and recovery rate are independent capabilities (Spearman ρ=0.126, p=0.747), indicating that model robustness to tool-use errors is at least two-dimensional.

RO3 (Mitigation).. A lightweight runtime interceptor reduces GPT-4o-mini hallucination by 23 percentage points under a concurrent n=600 control. The effect is model-dependent: Gemini-2.0-Flash shows no effect because its aggressive parameter rejection eliminates the target failure mode.

All code, data, 2,300 traces, and 100 human labels are released at https://github.com/bhaskargurram-ai/agenthallu-bench to support reproducible agent evaluation research.

Declarations

Competing Interests: None declared. Zasti Inc. is the author's employer; no external funder influenced the research.

Funding: Self-funded with employer support.

Data and Code: https://github.com/bhaskargurram-ai/agenthallu-bench.

References

[1] X. Liu, X. Yang, Z. Li, P. Li, R. He, AgentHallu: Benchmarking automated hallucination attribution of LLM-based agents, arXiv preprint (2026). URL: https://arxiv.org/abs/2601.06818, arXiv:2601.06818.

[2] S. Yao, N. Shinn, P. Razavi, K. Narasimhan, τ-bench: A benchmark for tool-agent-user interaction in real-world domains, arXiv preprint (2024). URL: https://arxiv.org/abs/2406.12045, arXiv:2406.12045.

[3] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, J. Tang, AgentBench: Evaluating LLMs as agents, in: Proceedings of the Twelfth International Conference on Learning Representations (ICLR), 2024. URL: https://arxiv.org/abs/2308.03688, arXiv:2308.03688.

[4] F. Yan, H. Mao, C. C.-J. Ji, T. Zhang, S. G. Patil, I. Stoica, J. E. Gonzalez, The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025. URL: https://openreview.net/forum?id=2GmDdhBdDk. doi:10.48550/arXiv.2502.19557.

[5] I. Soboroff, Don't use LLMs to make relevance judgments, arXiv preprint (2024). URL: https://arxiv.org/abs/2409.15133, arXiv:2409.15133.

[6] P. Thomas, S. Spielman, N. Craswell, B. Mitra, Large language models can accurately predict searcher preferences, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2024. URL: https://dl.acm.org/doi/10.1145/3626772.3657707. doi:10.1145/3626772.3657707.

[7] M. Alaofi, P. Thomas, F. Scholer, M. Sanderson, LLMs can be fooled into labelling a document as relevant, in: Proceedings of the 3rd International ACM SIGIR Conference on Information Retrieval in the Asia Pacific (SIGIR-AP), 2024. URL: https://dl.acm.org/doi/10.1145/3673791.3698410.

[8] V. Barres, H. Dong, S. Ray, X. Si, K. Narasimhan, τ^2-bench: Evaluating conversational agents in a dual-control environment, arXiv preprint (2025). URL: https://arxiv.org/abs/2506.07982, arXiv:2506.07982.

[9] MCPAgentBench Team, MCPAgentBench: A real-world task benchmark for evaluating LLM agent MCP tool use, arXiv preprint (2026). URL: https://arxiv.org/abs/2512.24565, arXiv:2512.24565.

[10] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, M. Shmueli-Scheuer, Survey on evaluation of LLM-based agents, arXiv preprint (2025). URL: https://arxiv.org/abs/2503.16416, arXiv:2503.16416.

[11] X. Xie, X. Li, H. Wang, Z. Yang, Q. Lv, L. Yu, A survey of large language model empowered agents for recommendation and search: Towards next-generation information retrieval, arXiv preprint (2025). URL: https://arxiv.org/abs/2503.05659, arXiv:2503.05659.

[12] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems 36 (NeurIPS), 2023. URL: https://arxiv.org/abs/2306.05685, arXiv:2306.05685.

[13] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, J. Guo, A survey on LLM-as-a-judge, arXiv preprint (2024). URL: https://arxiv.org/abs/2411.15594, arXiv:2411.15594.

[14] A. Bavaresco, A. Testoni, M. Poesio, S. Paun, A. Uma, T. Fornaciari, D. Hovy, B. Plank, R. Bernardi, LLMs instead of human judges? a large scale empirical study across 20 NLP evaluation tasks, arXiv preprint (2024). URL: https://arxiv.org/abs/2406.18403, arXiv:2406.18403.

[15] A. S. Thakur, K. Choudhary, V. Srinik, D. Hupkes, Judging the judges: Evaluating alignment and vulnerabilities in LLMs-as-judges, in: Findings of the Association for Computational Linguistics: ACL 2025, 2025. URL: https://aclanthology.org/2025.gem-1.33/, arXiv:2406.12624.

[16] X. Fu, et al., How reliable is multilingual LLM-as-a-judge?, in: Findings of the Association for Computational Linguistics: EMNLP 2025, 2025. URL: https://aclanthology.org/2025.findings-emnlp.587/, arXiv:2505.12434.

[17] K. Schroeder, Z. Wood-Doughty, Can you trust LLM judgments? reliability of LLM-as-a-judge, arXiv preprint (2024). URL: https://arxiv.org/abs/2412.12509, arXiv:2412.12509.

[18] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, ACM Transactions on Information Systems (2024). URL: https://arxiv.org/abs/2311.05232, arXiv:2311.05232.

[19] X. Lin, et al., LLM-based agents suffer from hallucinations: A survey of taxonomy, methods, and

directions, arXiv preprint (2025). URL: https://arxiv.org/abs/2509.18970, arXiv:2509.18970.

[20] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, J.-R. Wen, A survey on the memory mechanism of large language model-based agents, ACM Transactions on Information Systems (2025). URL: https://dl.acm.org/doi/10.1145/3748302. doi:10.1145/3748302.

[21] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: Proceedings of the Eleventh International Conference on Learning Representations (ICLR), 2023. URL: https://arxiv.org/abs/2203.11171, arXiv:2203.11171.

[22] P. Manakul, A. Liusie, M. J. F. Gales, SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. URL: https://arxiv.org/abs/2303.08896, arXiv:2303.08896.

[23] S. Farquhar, J. Kossen, L. Kuhn, Y. Gal, Semantic entropy for detecting confabulations in large language models, Nature (2024). URL: https://www.nature.com/articles/s41586-024-07421-0. doi:10.1038/s41586-024-07421-0.

[24] H. Wang, et al., AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents, in: Proceedings of the 48th International Conference on Software Engineering (ICSE), 2026. URL: https://arxiv.org/html/2503.18666v1, arXiv:2503.18666.

[25] K. Balog, D. Metzler, Z. Qin, Rankers, judges, and assistants: Towards understanding the interplay of LLMs in information retrieval evaluation, arXiv preprint (2025). URL: https://arxiv.org/abs/2503.19092, arXiv:2503.19092.

[26] A. Salemi, H. Zamani, Evaluating retrieval quality in retrieval-augmented generation, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2024. URL: https://dl.acm.org/doi/10.1145/3626772.3657957. doi:10.1145/3626772.3657957.

[27] H. A. Rahmani, N. Craswell, E. Yilmaz, B. Mitra, D. Campos, Synthetic test collections for retrieval evaluation, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2024. URL: https://dl.acm.org/doi/10.1145/3626772.3657942. doi:10.1145/3626772.3657942.

[28] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, J.-R. Wen, Large language models for information retrieval: A survey, arXiv preprint (2024). URL: https://arxiv.org/abs/2308.07107, arXiv:2308.07107.

[29] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, RAGAS: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association

for Computational Linguistics: System Demonstrations (EACL), 2024. URL: https://aclanthology.org/2024.eacl-demo.16/, arXiv:2309.15217.

[30] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, T. Zhang, RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024. URL: https://arxiv.org/abs/2401.00396, arXiv:2401.00396.

[31] J. R. Landis, G. G. Koch, The measurement of observer agreement for categorical data, Biometrics 33 (1977) 159–174. doi:10.2307/2529310.

[32] J. Cohen, A coefficient of agreement for nominal scales, Educational and Psychological Measurement 20 (1960) 37–46. doi:10.1177/001316446002000104.